# Backend Prompt

Here's the revised Antigravity prompt:

---

# MedNest — Backend Development Prompt

## Health Records System: Database + API Layer

---

## Context

You are building the backend for **MedNest**, a personal health management platform. This is a **Node.js + Express.js** backend inside a monorepo structured as:

```
mednest/
├── frontend/
├── backend/            ← You are working here
└── ml/                 → Does not exist yet, ignore completel
y
```

The backend's job right now:

1. Accept **image file uploads** (JPG/JPEG/PNG only) from the frontend
2. Store the raw file in Cloudinary
3. Create a structured database record
4. Serve structured, grouped, queryable data back to the frontend

**What is NOT being built right now:**

- ❌ Authentication (no JWT, no login, no protected routes — add later)
- ❌ ML pipeline (no Bull queue, no Redis, no Python service calls)
- ❌ AI summaries (no Claude API, no extraction logic)

Every schema field related to ML is still defined in the database — but nothing writes to it yet. The database must be **ML-ready from day one** even though the ML work happens later.

---

## Tech Stack

```
Runtime:         Node.js
Framework:       Express.js
Database:        MongoDB + Mongoose
File Storage:    Cloudinary
File Handling:   Multer
Config:          dotenv
Validation:      express-validator
Logging:         Morgan
```

---

## Folder Structure to Generate

```
backend/
├── config/
|   ├── db.js                  → MongoDB connection
|   └── cloudinary.js          → Cloudinary configuration
|
├── models/
|   ├── User.model.js          → Basic user schema (no auth m
ethods yet)
|   ├── HealthRecord.model.js  → Master record schema
|   └── MLJob.model.js         → ML job tracker (schema only,
not used yet)
|
├── routes/
|   └── records.routes.js
|
├── controllers/
|   └── records.controller.js
```

```
│
├── middleware/
│   ├── upload.middleware.js   → Multer + Cloudinary (JPG/PNG
only)
│   └── validate.middleware.js → Request validation
│
├── utils/
│   ├── groupRecords.js        → Smart grouping logic
│   ├── normalizeGroupKey.js   → Scan/lab type normalizer
│   └── ApiResponse.js         → Standardized response wrappe
r
│
├── seeds/
│   └── records.seed.js        → Sample data for testing
│
├── .env.example
├── app.js
└── server.js
```

---

## Environment Variables

Generate a complete `.env.example`:

```
# Server
PORT=8000
NODE_ENV=development

# MongoDB
MONGODB_URI=mongodb+srv://<user>:<password>@cluster.mongodb.n
et/mednest

# Cloudinary
CLOUDINARY_CLOUD_NAME=
CLOUDINARY_API_KEY=
CLOUDINARY_API_SECRET=
```

```
CLOUDINARY_RECORDS_FOLDER=mednest/records

# ML Service (not used now, reserved for later)
ML_SERVICE_URL=http://localhost:8001
```

---

## DATABASE SCHEMAS

### 1. `User.model.js`

Simple schema. No auth methods yet. Just the data structure.

```
{
  _id: ObjectId,

  fullName: {
    type: String,
    required: true,
    trim: true
  },
  email: {
    type: String,
    required: true,
    unique: true,
    lowercase: true
  },

  // Health profile (used by ML later for context)
  dateOfBirth: Date,
  gender: {
    type: String,
    enum: ['male', 'female', 'other']
  },
  bloodGroup: {
    type: String,
    enum: ['A+','A-','B+','B-','AB+','AB-','O+','O-']
```

```
    },
    heightCm: Number,
    weightKg: Number,

    profilePhoto: {
      url: String,
      cloudinaryId: String
    },

    emergencyContacts: [
      {
        name: String,
        relation: String,
        phone: String,
        isPrimary: Boolean
      }
    ],

    isActive: {
      type: Boolean,
      default: true
    }
  },
  { timestamps: true }
```

---

## 2. `HealthRecord.model.js`

This is the most critical schema. Every uploaded image becomes one `HealthRecord` document. The schema is fully ML-ready — all ML fields are defined and typed correctly — but nothing writes to them until the ML pipeline is built.

```
  {
    _id: ObjectId,

    // ── OWNERSHIP ──────────────────────────────────
```

```
  // No auth yet — userId will be passed manually in request
body
  // When auth is added later, this gets populated from JWT
  userId: {
    type: ObjectId,
    ref: 'User',
    required: true,
    index: true
  },

  // ── CLASSIFICATION ──────────────────────────
  // Set by user on upload
  // Later: ML will confirm or override this
  recordType: {
    type: String,
    enum: [
      'visit',
      'scan',
      'lab_report',
      'prescription',
      'consultation'
    ],
    required: true
  },

  subType: {
    type: String,
    // scan        → "Chest X-Ray", "MRI Brain", "USG Abdome
n", "ECG"
    // lab_report  → "Complete Blood Count", "Lipid Profile",
"Thyroid Panel"
    // visit       → "General Checkup", "Follow-up", "Emergen
cy"
    // consultation→ "Cardiology", "Neurology", "Dermatology"
    // prescription→ "General", "Specialist"
  },
```

```
// Normalized version of subType
// Generated by normalizeGroupKey utility on every save
// Used for smart grouping queries
groupKey: {
  type: String,
  index: true
},

// ── METADATA ──────────────────────────────────
// User provides this on upload
// Later: ML will extract and correct from image
recordDate: {
  type: Date,
  required: true
},

doctorName: {
  type: String,
  default: null
},
facilityName: {
  type: String,
  default: null
},
userNotes: {
  type: String,
  default: null
},

// ── FILE STORAGE ──────────────────────────────
// JPG / JPEG / PNG only
originalFile: {
  cloudinaryId: {
    type: String,
    required: true
```

```
    },
    url: {
      type: String,
      required: true
    },
    fileType: {
      type: String,
      enum: ['jpg', 'jpeg', 'png'],
      required: true
    },
    fileSizeKb: Number,
    originalFileName: String,
    width: Number,
    height: Number
    // Cloudinary returns image dimensions on upload
    // Store them — useful for scan image display later
  },

  // ── ML PIPELINE STATE ──────────────────────────
  // This entire block reserved for ML pipeline
  // Default state: pending
  // Nothing writes to this yet
  mlPipeline: {
    status: {
      type: String,
      enum: ['pending', 'processing', 'completed', 'failed'],
      default: 'pending'
    },
    jobId: {
      type: String,
      default: null
    },
    startedAt: {
      type: Date,
      default: null
    },
```

```
    completedAt: {
      type: Date,
      default: null
    },
    failedAt: {
      type: Date,
      default: null
    },
    failureReason: {
      type: String,
      default: null
    },
    retryCount: {
      type: Number,
      default: 0
    },
    mlServiceVersion: {
      type: String,
      default: null
    },
    confidence: {
      classification: {
        type: Number,
        default: null
      },
      extraction: {
        type: Number,
        default: null
      }
    }
  },

  // ── EXTRACTED DATA ──────────────────────────────
  // Reserved for ML pipeline output
  // ML will populate this with structured data
  // after processing the image
```

```
  // Structure varies by recordType
  // See extractedData shapes section
  extractedData: {
    type: mongoose.Schema.Types.Mixed,
    default: null
  },

  // ── AI SUMMARY ──────────────────────────────
  // Reserved for ML pipeline + Claude API
  aiSummary: {
    type: String,
    default: null
  },
  keyFindings: {
    type: [String],
    default: []
  },

  // ── USER OVERRIDES ──────────────────────────
  // Tracks if user manually corrected ML extraction
  userOverrides: {
    type: Boolean,
    default: false
  },
  originalMlExtraction: {
    type: mongoose.Schema.Types.Mixed,
    default: null
    // When user edits extractedData, original ML output
    // is preserved here for comparison
  },

  // ── COMPARISON ──────────────────────────────
  comparedWith: [
    {
      type: ObjectId,
      ref: 'HealthRecord'
```

```
    }
  ],

  // ── DUPLICATE DETECTION ──────────────────────
  // Reserved for ML duplicate detection
  duplicateDetection: {
    isDuplicate: {
      type: Boolean,
      default: false
    },
    duplicateOf: {
      type: ObjectId,
      ref: 'HealthRecord',
      default: null
    },
    similarityScore: {
      type: Number,
      default: null
    }
  },

  // ── SOFT DELETE ──────────────────────────────
  isDeleted: {
    type: Boolean,
    default: false,
    index: true
  },
  deletedAt: {
    type: Date,
    default: null
  }
},
{ timestamps: true }
```

**Indexes:**

```
HealthRecordSchema.index({ userId: 1, recordType: 1, recordDa
te: -1 })
HealthRecordSchema.index({ userId: 1, groupKey: 1, recordDat
e: 1 })
HealthRecordSchema.index({ userId: 1, isDeleted: 1, recordDat
e: -1 })
HealthRecordSchema.index({ userId: 1, 'mlPipeline.status': 1
})
```

**Pre-save hook:**

```
// Before every save, auto-generate groupKey from subType
HealthRecordSchema.pre('save', function(next) {
  if (this.subType) {
    this.groupKey = normalizeGroupKey(this.subType)
  }
  next()
})
```

---

## 3. `MLJob.model.js`

Schema defined now, not used until ML pipeline is built.

```
{
  _id: ObjectId,

  recordId: {
    type: ObjectId,
    ref: 'HealthRecord',
    required: true
  },
  userId: {
    type: ObjectId,
    ref: 'User',
    required: true
```

```
  },

  bullJobId: {
    type: String,
    default: null
  },
  status: {
    type: String,
    enum: ['queued', 'processing', 'completed', 'failed', 're
trying'],
    default: 'queued'
  },

  mlInput: {
    fileUrl: String,
    fileType: String,
    recordTypeHint: String,
    userId: String
  },

  // Full raw JSON response from Python ML service
  // Stored for debugging
  mlOutput: {
    type: mongoose.Schema.Types.Mixed,
    default: null
  },

  queuedAt: Date,
  processingStartedAt: Date,
  completedAt: Date,
  processingDurationMs: Number,

  error: {
    message: String,
    stack: String,
    mlServiceResponse: mongoose.Schema.Types.Mixed
```

```
    },

    retryCount: {
      type: Number,
      default: 0
    }
  },
  { timestamps: true }
```

---

## EXTRACTED DATA SHAPES

Document these as JSDoc comments inside `HealthRecord.model.js`. These are the shapes the ML service will populate into `extractedData` later. Defining them now ensures the database and frontend are built to accommodate them correctly.

### Lab Report

```
extractedData: {
  labName: String,
  accreditationNumber: String,
  sampleCollectedAt: Date,
  reportedAt: Date,
  orderedBy: String,
  parameters: [
    {
      name: String,
      parameterKey: String,
      // Normalized: "hemoglobin", "wbc_count"
      // Used for trend queries across multiple reports
      value: Number,
      unit: String,
      referenceMin: Number,
      referenceMax: Number,
      status: String
      // 'normal' | 'low' | 'high' |
```

```
      // 'borderline_low' | 'borderline_high' |
      // 'critical_low' | 'critical_high'
    }
  ]
}
```

## Scan

```
extractedData: {
  scanType: String,
  bodyRegion: String,
  view: String,
  modalityType: String,
  // "X-Ray" | "MRI" | "CT" | "Ultrasound" | "ECG"
  radiologistName: String,
  radiologistCredentials: String,
  scanPerformedAt: Date,
  reportDate: Date,
  findings: [String],
  impression: String,
  recommendations: String,
  scanImageUrl: String,
  // Same as originalFile.url in most cases
  // Separate field in case a processed/enhanced
  // version is stored later
  isReportAttached: Boolean,
  contrastUsed: Boolean,
  urgency: String
  // 'routine' | 'urgent' | 'stat'
}
```

## Prescription

```
extractedData: {
  diagnosis: String,
```

```
  doctorRegNumber: String,
  prescriptionDate: Date,
  isHandwritten: Boolean,
  medications: [
    {
      brandName: String,
      genericName: String,
      drugClass: String,
      dosage: String,
      frequency: String,
      frequencyCode: String,
      // "OD" | "BD" | "TDS" | "QID" | "SOS"
      duration: String,
      durationDays: Number,
      form: String,
      // 'tablet'|'capsule'|'syrup'|'injection'
      // 'cream'|'drops'|'inhaler'|'patch'|'other'
      instructions: String,
      refillsAllowed: Number,
      isControlled: Boolean
    }
  ],
  specialInstructions: String,
  followUpDate: Date
}
```

## Visit

```
extractedData: {
  visitType: String,
  chiefComplaint: String,
  diagnosis: [String],
  vitals: {
    bloodPressureSystolic: Number,
    bloodPressureDiastolic: Number,
```

```
    heartRateBpm: Number,
    temperatureCelsius: Number,
    weightKg: Number,
    heightCm: Number,
    bmi: Number,
    spo2Percent: Number,
    respiratoryRate: Number
  },
  doctorNotes: String,
  treatmentGiven: String,
  referrals: [String],
  testsOrdered: [String],
  nextAppointmentDate: Date
}
```

## Consultation

```
extractedData: {
  specialty: String,
  mode: String,
  // 'in_person' | 'teleconsultation' | 'home_visit'
  referredBy: String,
  chiefComplaint: String,
  clinicalAssessment: String,
  diagnosis: [String],
  recommendations: [String],
  testsOrdered: [String],
  medicationsPrescribed: [String],
  followUpInstructions: String,
  followUpDate: Date,
  urgency: String
  // 'routine' | 'urgent' | 'emergency'
}
```

## UPLOAD MIDDLEWARE

`middleware/upload.middleware.js`

```
// Multer configuration:
// - Storage: memory storage (buffer, not disk)
// - Accepted MIME types: image/jpeg, image/jpg, image/png ON
LY
// - Max file size: 10MB
// - Single file per request, field name: "file"
// - On invalid type: throw ApiError 400
//   "Only JPG, JPEG, and PNG files are accepted"
// - On size exceeded: throw ApiError 400
//   "File size must be under 10MB"

// After Multer, Cloudinary upload function:
// - Upload from buffer using upload_stream
// - Folder: process.env.CLOUDINARY_RECORDS_FOLDER + "/" + us
erId
// - Resource type: "image"
// - Allowed formats: ["jpg", "jpeg", "png"]
// - On success: attach to req.cloudinaryResult:
//   {
//     cloudinaryId: result.public_id,
//     url: result.secure_url,
//     fileType: detected from mimetype,
//     fileSizeKb: Math.round(file.size / 1024),
//     originalFileName: file.originalname,
//     width: result.width,
//     height: result.height
//   }
```

---

## API ROUTES — Full Specification

Base URL: `/api/v1/records`

No authentication middleware on any route right now. When auth is added later, `authMiddleware` slots in before the controller on every route.

---

## 1. UPLOAD A RECORD

```
POST /upload

Middleware: multer → cloudinaryUpload

Body: multipart/form-data
  file         File     required   JPG/JPEG/PNG image
  userId       String   required   Hardcoded for now, JWT lat
er
  recordType   String   required   enum: visit|scan|lab_repor
t|
                                        prescription|consulta
tion
  subType      String   optional   "Chest X-Ray", "CBC", etc.
  recordDate   String   required   ISO date string
  doctorName   String   optional
  facilityName String   optional
  userNotes    String   optional

Validation:
  - file must be present
  - userId must be a valid MongoDB ObjectId
  - recordType must be one of the enum values
  - recordDate must be a valid date, not in the future

Flow:
  1. Validate all fields
  2. File uploaded to Cloudinary by middleware
  3. Generate groupKey from subType using normalizeGroupKey()
  4. Create HealthRecord:
     {
```

```
        userId,
        recordType,
        subType,
        groupKey,
        recordDate,
        doctorName,
        facilityName,
        userNotes,
        originalFile: req.cloudinaryResult,
        mlPipeline: { status: 'pending' }
        // All ML fields default to null/pending
        // Ready for ML pipeline to populate later
      }
  5. Save to MongoDB
  6. Return created record

Response 201:
  {
    success: true,
    message: "Record uploaded successfully",
    data: {
      record: HealthRecord
    }
  }
```

---

## 2. GET ALL RECORDS (Timeline)

```
GET /

Query params:
  userId     String   required (hardcoded now, JWT later)
  type       String   optional filter by recordType
  groupBy    Boolean  optional default: false
  from       String   optional ISO date
  to         String   optional ISO date
```

```
  search     String   optional searches doctorName,
                               facilityName, subType
  page       Number   optional default: 1
  limit      Number   optional default: 20

Always exclude: isDeleted = true

Response (groupBy=false — flat timeline, sorted newest firs
t):
  {
    success: true,
    data: {
      records: [HealthRecord],
      pagination: {
        total: Number,
        page: Number,
        limit: Number,
        totalPages: Number,
        hasNextPage: Boolean,
        hasPrevPage: Boolean
      }
    }
  }

Response (groupBy=true — smart grouped by groupKey):
  {
    success: true,
    data: {
      groups: [
        {
          groupKey: String,
          subType: String,
          recordType: String,
          count: Number,
          latestDate: Date,
          earliestDate: Date,
```

```
          records: [HealthRecord]
          // Sorted oldest → newest inside group
          // So frontend can render the mini-timeline
        }
      ]
      // Groups sorted by latestDate descending
    }
  }
```

---

## 3. GET SINGLE RECORD

```
GET /:recordId

Validation:
  recordId must be valid ObjectId
  record must not be isDeleted

Response 200:
  {
    success: true,
    data: {
      record: HealthRecord  // Full document
    }
  }

Response 404:
  {
    success: false,
    message: "Record not found"
  }
```

---

## 4. UPDATE A RECORD

```
PUT /:recordId

Body (all fields optional):
  {
    recordType:   String,
    subType:      String,
    recordDate:   String,
    doctorName:   String,
    facilityName: String,
    userNotes:    String,
    extractedData: Object
    // User manually filling in extracted data
    // OR correcting what ML will return later
  }

When extractedData is updated:
  If extractedData already has ML-populated data:
    - Copy current extractedData → originalMlExtraction
    - Set userOverrides = true
    - Set new extractedData to what user provided
  If no ML data exists yet (null):
    - Just set extractedData directly
    - userOverrides = true

If subType is updated:
  Regenerate groupKey using normalizeGroupKey(newSubType)

Response 200:
  {
    success: true,
    message: "Record updated",
    data: {
      record: HealthRecord
```

```
    }
  }
```

## 5. DELETE A RECORD

```
DELETE /:recordId

Soft delete only — never hard delete from database

Flow:
  1. Set isDeleted = true
  2. Set deletedAt = now
  3. Delete image from Cloudinary using cloudinaryId
     (hard delete from Cloudinary is fine)

Response 200:
  {
    success: true,
    message: "Record deleted successfully"
  }
```

## 6. GET GROUPED SCANS

```
GET /grouped/scans

Query params:
  userId     String   required

Returns all scan records grouped by groupKey
Specifically for the scan timeline view on the frontend

Response:
  {
    success: true,
```

```
    data: {
      groups: [
        {
          groupKey: String,
          // "chest_xray", "mri_brain", "usg_abdomen"
          subType: String,
          // "Chest X-Ray", "MRI Brain", "USG Abdomen"
          count: Number,
          latestDate: Date,
          earliestDate: Date,
          records: [
            {
              _id: ObjectId,
              recordDate: Date,
              doctorName: String,
              facilityName: String,
              originalFile: { url, fileType },
              mlPipeline: { status },
              extractedData: Mixed,
              aiSummary: String,
              createdAt: Date
            }
          ]
        }
      ]
    }
  }
```

---

## 7. GET PARAMETER TRENDS

```
GET /trends/:parameterKey

Query params:
  userId       String   required
  recordType   String   required
```

```
  from         String   optional ISO date
  to           String   optional ISO date

Example:
  GET /trends/hemoglobin?userId=xxx&recordType=lab_report

Logic:
  Find all HealthRecords where:
    userId matches
    recordType matches
    extractedData.parameters has entry where
    parameterKey = req.params.parameterKey
    isDeleted = false
    mlPipeline.status = 'completed'

Response:
  {
    success: true,
    data: {
      parameterKey: String,
      unit: String,
      referenceMin: Number,
      referenceMax: Number,
      dataPoints: [
        {
          recordId: ObjectId,
          date: Date,
          value: Number,
          status: String
        }
      ]
      // Sorted oldest → newest for charting
    }
  }
```

## 8. COMPARE TWO RECORDS

```
POST /compare

Body:
  {
    recordIdA: String,
    recordIdB: String
  }

Validation:
  Both must be valid ObjectIds
  Both must exist and not be deleted

Current behavior (no ML yet):
  Fetch both records
  Return them side by side
  No delta computation yet
  Delta will be computed by ML service later

Response:
  {
    success: true,
    data: {
      recordA: HealthRecord,
      recordB: HealthRecord,
      // ML delta will be added here later:
      comparison: null
      // When ML is ready:
      // comparison: {
      //   summary: String,
      //   changes: [...],
      //   overallTrend: String,
      //   aiInsight: String
      // }
```

```
    }
  }
```

---

## UTILITY: `normalizeGroupKey.js`

Build as a lookup map with aliases. The ML service will also use this same logic in Python — keeping both in sync is important.

```
const ALIASES = {
  // X-Ray variants
  'chest x-ray': 'chest_xray',
  'x ray chest': 'chest_xray',
  'cxr': 'chest_xray',
  'chest xray': 'chest_xray',

  // Ultrasound variants
  'usg abdomen': 'usg_abdomen',
  'ultrasound abdomen': 'usg_abdomen',
  'abdominal ultrasound': 'usg_abdomen',
  'usg upper abdomen': 'usg_abdomen',

  // ECG variants
  'ecg': 'ecg',
  'ekg': 'ecg',
  'electrocardiogram': 'ecg',
  'electrocardiograph': 'ecg',

  // MRI variants
  'mri brain': 'mri_brain',
  'brain mri': 'mri_brain',
  'mri head': 'mri_brain',

  // CT variants
  'ct chest': 'ct_chest',
  'chest ct': 'ct_chest',
```

```
  'hrct chest': 'ct_chest',

  // Lab variants
  'complete blood count': 'cbc',
  'cbc': 'cbc',
  'full blood count': 'cbc',

  'lipid profile': 'lipid_profile',
  'lipid panel': 'lipid_profile',
  'cholesterol test': 'lipid_profile',

  'thyroid panel': 'thyroid_panel',
  'thyroid profile': 'thyroid_panel',
  'tft': 'thyroid_panel',
  'thyroid function test': 'thyroid_panel',

  'blood glucose': 'blood_glucose',
  'fasting blood sugar': 'blood_glucose',
  'fbs': 'blood_glucose',
  'rbs': 'blood_glucose',
  'hba1c': 'hba1c',

  'liver function test': 'lft',
  'lft': 'lft',

  'kidney function test': 'kft',
  'renal function test': 'kft',
  'kft': 'kft',
  'rft': 'kft',
}

// Fallback: lowercase + trim + replace spaces with underscor
e
function normalizeGroupKey(subType) {
  if (!subType) return null
  const key = subType.toLowerCase().trim()
```

```
  return ALIASES[key] || key.replace(/\s+/g, '_')
}
```

## UTILITY: `ApiResponse.js`

```
class ApiResponse {
  constructor(statusCode, data, message = "Success") {
    this.statusCode = statusCode
    this.data = data
    this.message = message
    this.success = statusCode < 400
  }
}

class ApiError extends Error {
  constructor(statusCode, message, errors = []) {
    super(message)
    this.statusCode = statusCode
    this.data = null
    this.message = message
    this.success = false
    this.errors = errors
    Error.captureStackTrace(this, this.constructor)
  }
}
```

## SEED FILE: `seeds/records.seed.js`

Populate with the following hardcoded data. Use a fixed userId for all records during development.

```
// Development userId (replace with real ObjectId from your U
ser collection)
const DEV_USER_ID = "000000000000000000000001"
```

```
// Records to seed — mirrors the MedNest sample data:

2025 March:
  - Visit: General Checkup | Dr. Priya Sharma | Apollo Hospit
al | Mar 12 2025
  - Lab: Complete Blood Count | Mar 12 2025
  - Scan: ECG | Mar 12 2025

2025 January:
  - Prescription: Amlodipine 5mg | Dr. Rajan Mehta | Jan 8 20
25
  - Consultation: Cardiology | Dr. Rajan Mehta | Fortis Hospi
tal | Jan 8 2025

2024 August:
  - Scan: Chest X-Ray | Dr. K. Venkatesh | Apollo Radiology |
Aug 3 2024
  - Scan: USG Abdomen | Aug 3 2024
  - Lab: Lipid Profile | Aug 3 2024

2024 April:
  - Scan: MRI Brain | Apr 18 2024
  - Consultation: Neurology | Dr. Kavitha Rao | NIMHANS | Apr
18 2024

2024 January:
  - Scan: Chest X-Ray | Jan 22 2024
  - Scan: ECG | Jan 22 2024
  - Prescription: Atorvastatin 10mg | Dr. Rajan Mehta | Jan 2
2 2024

2023 October:
  - Lab: Thyroid Panel | Oct 5 2023
  - Prescription: Levothyroxine 50mcg | Dr. Priya Sharma | Oc
t 5 2023
```

```
2023 June:
  - Scan: USG Abdomen | Jun 14 2023
  - Visit: Follow-up | Dr. Priya Sharma | Apollo Hospital | J
un 14 2023
  - Scan: ECG | Jun 14 2023

2023 February:
  - Lab: Complete Blood Count | Feb 28 2023
  - Lab: Lipid Profile | Feb 28 2023
  - Visit: Annual Health Checkup | Dr. Priya Sharma | Apollo
Hospital | Feb 28 2023

// All records: mlPipeline.status = 'pending'
// All records: extractedData = null
// All records: aiSummary = null
// Use placeholder image URL for originalFile.url
// (any public JPG from Cloudinary or placeholder service)
```

---

## WHAT TO BUILD IN ORDER

```
Step 1 — Project Setup
  ✓ package.json with all dependencies
  ✓ app.js — Express setup, CORS, Morgan, JSON parser
  ✓ server.js — listens on PORT
  ✓ config/db.js — Mongoose connect
  ✓ config/cloudinary.js — SDK init
  ✓ .env.example

Step 2 — Models
  ✓ User.model.js
  ✓ HealthRecord.model.js (full schema + indexes + pre-save h
ook)
  ✓ MLJob.model.js (schema only)
```

```
Step 3 — Utilities
  ✓ ApiResponse.js + ApiError class
  ✓ normalizeGroupKey.js

Step 4 — Middleware
  ✓ upload.middleware.js (Multer + Cloudinary, JPG/PNG only)
  ✓ validate.middleware.js

Step 5 — Records API
  ✓ records.routes.js (all 8 routes wired)
  ✓ records.controller.js (all 8 handlers)

Step 6 — Seed
  ✓ seeds/records.seed.js
  ✓ npm run seed script in package.json

Step 7 — Test
  ✓ All routes testable via Postman
  ✓ Upload a real JPG and verify it appears in MongoDB + Clou
dinary
  ✓ Verify smart grouping returns correct clusters
```

---

## CRITICAL NOTES FOR ANTIGRAVITY

- No authentication on any route right now. `userId` is passed in request body/query params temporarily. When JWT is added later, it replaces this without any schema changes.
- The `mlPipeline` block, `extractedData`, `aiSummary`, and `keyFindings` are all defined in the schema but **nothing writes to them yet**. They default to `pending` / `null`. This is intentional.
- File types are **strictly JPG/JPEG/PNG only**. Reject everything else with a clear 400 error.

- Smart grouping uses MongoDB aggregation pipeline, not JavaScript-level grouping.
- Every controller function must be wrapped in `asyncHandler` (try/catch wrapper) — no unhandled promise rejections.
- All responses follow the `ApiResponse` wrapper format without exception.
- `normalizeGroupKey` runs on every save via pre-save hook — never call it manually in controllers.